

Overview of Adam optimizer

Subject: Adam optimizer

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

1.

where the parameter w which minimizes $Q(w)$ is to be estimated, η is a step size (sometimes called the learning rate in machine learning) and α is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w , thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.

2.

The step size is denoted by η (sometimes called the learning rate in machine learning) and here $:=$ denotes the update of a variable in the algorithm. In many cases, the summand functions have a simple form that enables inexpensive evaluations of the sum-function and the sum gradient. For example, in statistics, one-parameter exponential families allow economical function-evaluations and gradient-evaluations. However, in other cases, evaluating the sum-gradient may require expensive evaluations of the gradients from all summand functions. When the training set is enormous and no simple formulas exist, evaluating the sums of gradients becomes very expensive, because evaluating the gradient requires evaluating all the summand functions' gradients. To economize on the computational cost at every iteration, stochastic gradient descent samples a subset of summand functions at every step. This is very effective in the case of large-scale machine learning problems.

3.

Notable applications Stochastic gradient descent is a popular algorithm for training a wide range of models in machine learning, including (linear) support vector machines, logistic regression (see, e.g., Vowpal Wabbit) and graphical models. When combined with the backpropagation algorithm, it is the de facto standard algorithm for training artificial neural networks. Its use has been also reported in the

Geophysics community, specifically to applications of Full Waveform Inversion (FWI). Stochastic gradient descent competes with the L-BFGS algorithm, which is also widely used. Stochastic gradient descent has been used since at least 1960 for training linear regression models, originally under the name ADALINE. Another stochastic gradient descent algorithm is the least mean squares (LMS) adaptive filter.

4.

$$m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla_w L(t-1) \quad \{\displaystyle m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla_w L(t-1)\}$$

5.

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

6.

Approximations in continuous time For small learning rate η stochastic gradient descent $(w_n)_{n \in \mathbb{N}_0}$ can be viewed as a discretization of the gradient flow ODE

7.

Second-order methods A stochastic analogue of the standard (deterministic) Newton–Raphson algorithm (a "second-order" method) provides an asymptotically optimal or near-optimal form of iterative optimization in the setting of stochastic approximation. A method that uses direct

measurements of the Hessian matrices of the summands in the empirical risk function was developed by Byrd, Hansen, Nocedal, and Singer. However, directly determining the required Hessian matrices for optimization may not be possible in practice. Practical and theoretically sound methods for second-order versions of SGD that do not require direct Hessian information are given by Spall and others. (A less efficient method based on finite differences, instead of simultaneous perturbations, is given by Ruppert.) Another approach to the approximation Hessian matrix is replacing it with the Fisher information matrix, which transforms usual gradient to natural. These methods not requiring direct Hessian information are based on either values of the summands in the above empirical risk function or values of the gradients of the summands (i.e., the SGD inputs). In particular, second-order optimality is asymptotically achievable without direct calculation of the Hessian matrices of the summands in the empirical risk function. When the objective is a nonlinear least-squares loss